

W16 Assignment

Task 3: Agentify the Assistant

Objective

Extend the assistant you built in W15 by adding a feature that requires an **agentic loop**. The model should be able to decide what action to take next based on the result of the previous step, rather than following a fixed sequence of application-defined steps.

Your implementation should also demonstrate the **context engineering** and **multi-agent design** principles discussed in class.

Core Functionality

1. Add an Agentic Feature

Add at least one new feature to your W15 assistant that cannot be handled effectively by a fixed, single-pass pipeline.

Do not simply place your existing W15 RAG pipeline inside a loop. The new feature should require the model to evaluate intermediate results and decide what to do next.

You may choose one of the following directions, modify one of them, or propose your own:

- **Cross-source verification:** Answer a query by checking multiple sources or tools. The assistant should decide whether the available evidence is sufficient and perform another search when necessary.
- **Self-check before responding:** Generate an initial answer, verify it against the available sources, and revise it if the verification fails.
- **Open-ended comparison or research:** For example, compare several options and recommend one. The number and order of tool calls should depend on what the assistant discovers.
- **Multi-turn task completion:** Handle a task across multiple exchanges while tracking completed steps, missing information, and whether the task is finished.

Before implementation, write **one sentence explaining why a fixed pipeline would not be sufficient for your chosen feature**.

If you cannot clearly explain this, reconsider your feature.

2. Agentic Loop Requirements

Your loop must:

- Be capable of running for more than one iteration for a request.

- Allow the model to decide whether it should search again, use another tool, or ask the user for clarification.
- Have a clearly defined stopping condition, such as a maximum number of iterations or steps.

The loop must not be allowed to run indefinitely.

Context Engineering

Apply at least **one context-engineering technique** discussed in class. Use the technique where it provides a clear benefit to your agentic workflow.

Possible techniques include:

- Clearing tool results
- Capping or re-ranking retrieval results
- Compaction
- Structured external notes
- Using a sub-agent for verbose exploration
- **Progressive disclosure through Skills:** Load a concise **SKILL.md** description into the context initially and provide the full instructions only when the model determines that the skill is relevant.

Document which technique you used and how it affects your system.

Documentation Requirements

The README must include the following sections. These sections are part of the assessment and should be based on your implementation rather than general explanations of concepts from class.

a. Context Engineering Technique

Explain:

1. Which technique you used.
2. Where it is applied in your agentic loop.
3. What problem it solves.

For example, instead of writing *"I used compaction,"* explain what was causing the context problem and why compaction was introduced at that point in the workflow.

b. Agentic Pattern

State whether your implementation uses:

- A **single-agent loop**, or
- A **multi-agent system**.

Then explain why you made that choice using the frameworks discussed in class.

If you chose a multi-agent system, relate your decision to one or more of the following benefits:

- Context isolation
- Parallelization
- Specialization

You may also use the **five structural failures** framework:

- Context saturation
- Sequential bottleneck
- Skill dilution
- Self-verification paradox
- Single point of failure

Using multiple agents does not automatically make the system better. A single-agent design is completely acceptable when it is the better fit for the task.

c. Evaluation Harness

Build an evaluation harness **from scratch**. Do not use an existing evaluation framework.

Your harness should test the actual behavior of your new agentic feature.

At minimum, measure:

- **Task completion rate:** How often does the agent successfully complete the task across your test queries?
- **Tool-call correctness:** Does the agent select the appropriate tool and provide valid arguments?
- **Trajectory length:** How many iterations does each query require? Is the number reasonable given the complexity of the query?
- **Failure log:** Record unsuccessful cases and classify them as:
 - Hard failure
 - Soft failure
 - Cascading soft failure

Use the failure taxonomy discussed in class.

Additional Requirements

1. Skill vs. Agent

Before adding a new agent or tool, write **one sentence explaining whether the capability could instead have been implemented as a Skill**.

If a Skill would have been sufficient, explain why you chose not to use one.

2. Token and Cost Accounting

The evaluation harness must record the **total number of tokens consumed for each query**.

If you implemented a multi-agent system, compare its token usage with a single-agent baseline when possible. The goal is to make the additional coordination cost visible in your evaluation results.

3. Failure Injection Test

Intentionally introduce one failure into the system. For example:

- Make a tool unavailable.
- Provide malformed retrieval output.
- Force a timeout.

Document how the agent responds.

Pay particular attention to whether the system recognizes the failure and responds appropriately, rather than producing a confident answer based on invalid or incomplete information.

4. Tool vs. Agent Boundary

If your assistant uses an external service that is itself multi-step or stateful, explain in one paragraph how you modeled it:

- As a bounded tool call, or
- As an agent-to-agent interaction.

Explain why you made that design choice.

No additional implementation is required for this part.

5. Write-up Length

Keep sections a–c and the four additional requirements above to approximately one page.

This section should explain the decisions you made in your own system. Do not use it to repeat the course material.

Deliverables

Submit the following:

1. **Updated source code**
 - Your W15 assistant
 - The new agentic feature
2. **Updated README**

- Documentation Requirements (a, b, c)
- Additional Requirements
- 3. **Updated architecture diagram**
 - Show the agentic loop
 - If using multiple agents, show the coordination structure
- 4. **Evaluation harness**
 - Source code
 - Results report
 - A simple table or Markdown output is sufficient